

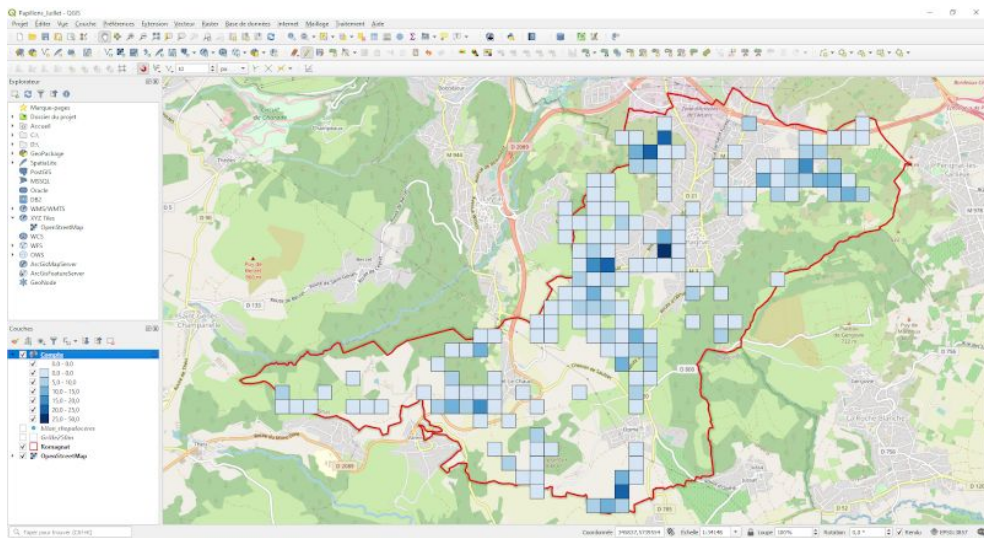
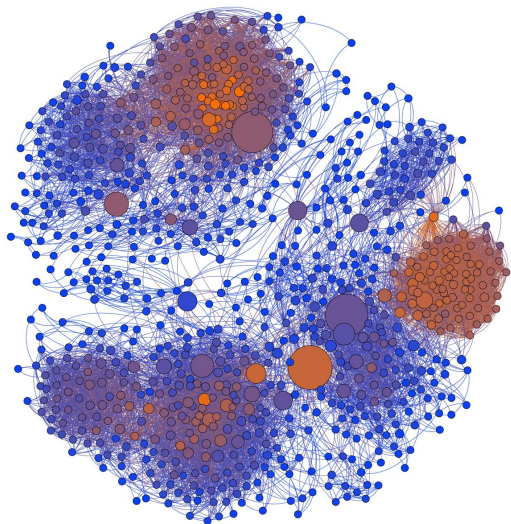


Initiation à Python

(pour les SHS, mais pas que)

Calcul scientifique appliqué aux données archéologiques et géohistoriques
UMR 7041 - ArScAn





Qu'est-ce que programmer ?

Avant de parler de Python, une petite question :

Pour vous, c'est quoi “programmer” ?

Pas besoin de définition savante. Un mot, une idée, une image suffit.

Automatiser ?

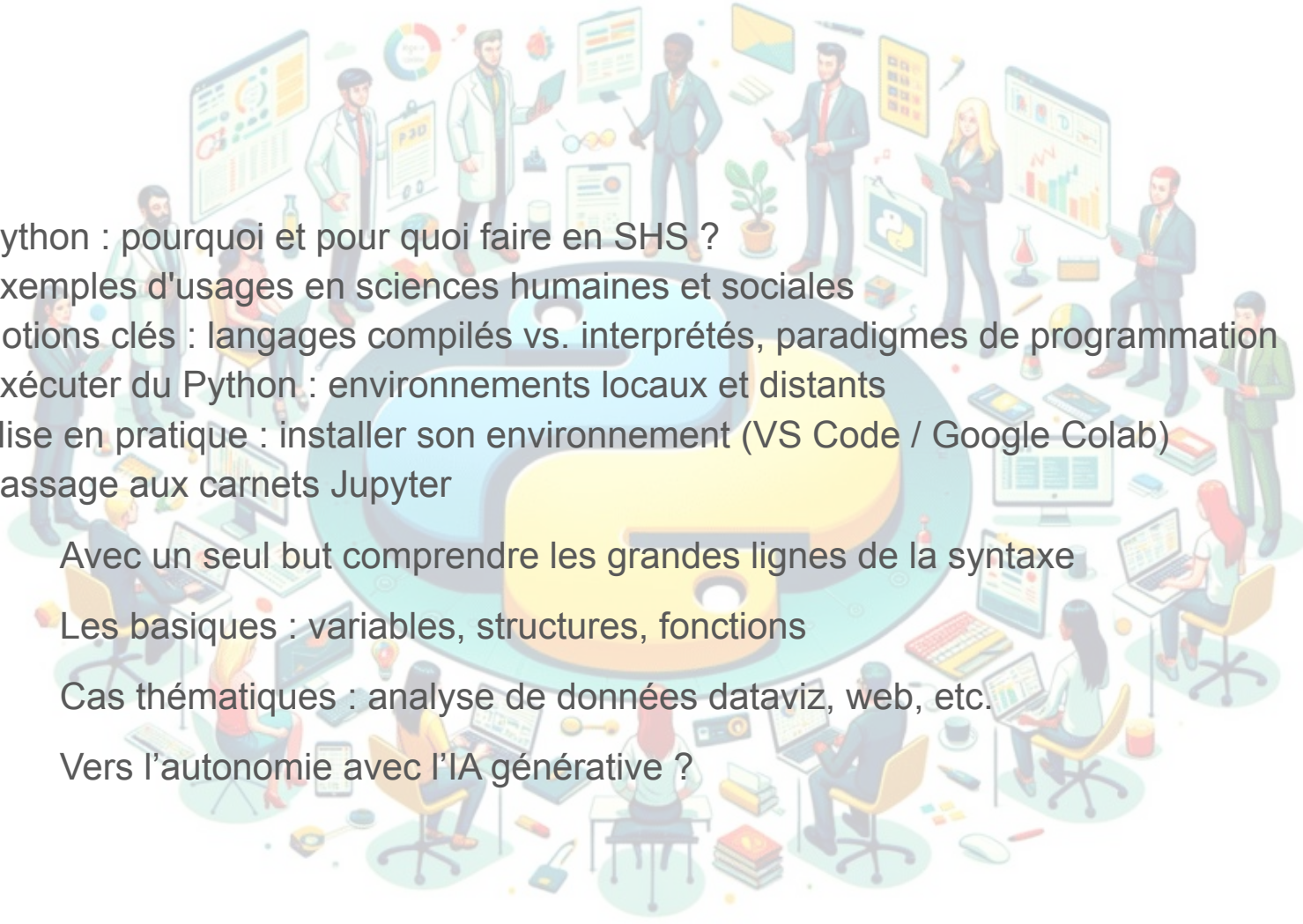
Donner des ordres à une machine ?

Résoudre un problème ?

Faire des calculs ?

Créer une application ?

Écrire des trucs incompréhensibles dans un terminal noir ?

- 
1. Python : pourquoi et pour quoi faire en SHS ?
 2. Exemples d'usages en sciences humaines et sociales
 3. Notions clés : langages compilés vs. interprétés, paradigmes de programmation
 4. Exécuter du Python : environnements locaux et distants
 5. Mise en pratique : installer son environnement (VS Code / Google Colab)
 6. Passage aux carnets Jupyter

Avec un seul but comprendre les grandes lignes de la syntaxe

Les basiques : variables, structures, fonctions

Cas thématiques : analyse de données dataviz, web, etc.

Vers l'autonomie avec l'IA générative ?

Automatisation de tâches répétitives

Inventaires

CSV / Excel

APIs / web scraping.

Analyse de données et

statistiques :

traiter, manipuler et analyser
des jeux de données



Programmation Web :
Développement de
sites web ou
d'applications web

Développement
d'outils pour la gestion
documentaire

Manipulation et
analyse de fichiers
multimédias

Python pour les SHS. Pourquoi ? Comment ?

Corpus textuels & linguistique

- Nettoyage/normalisation de texte, OCR et correction d'erreurs OCR
- Analyse de corpus (NLP) : lemmatisation, étiquetage morpho-syntaxique, reconnaissance d'entités nommées (personnes, lieux, dates)
- Fouille de texte, topic modeling, similarité sémantique
- Analyse de sentiments
- Parsing de corpus structurés (XML/TEI)

Données géospatiales, géohistoire, archéologie

- SIG en Python (geopandas, PyQGIS), géocodage d'adresses historiques
- Géoréférencement et tuilage de cartes anciennes
- Analyse spatiale, plus courts chemins sur réseaux historiques (cf. Cassinet)

Analyse de réseaux

- Réseaux de correspondance, de collaboration, de parenté
- Visualisation de graphes, détection de communautés, centralité

Statistiques et données quantitatives

- Manipulation de tableaux de données (pandas)
- Statistiques descriptives/inférentielles, analyses multivariées (ACP, AFC)
- Visualisation de données

Gestion de données de recherche

- Automatisation de tâches répétitives (inventaires, catalogues)
- Interrogation de bases avec expressions régulières
- Nettoyage et homogénéisation de sources hétérogènes (Excel, CSV, bases patrimoniales)

Web et interopérabilité

- Interrogation d'API (Wikipedia, Wikidata, Gallica, IIIF...)
- Web scraping de sources numérisées
- Développement de petites applications de diffusion de la recherche

Reproductibilité et diffusion

- Notebooks Jupyter : reproductibilité, script partageable, support pédagogique
- Publication scientifique appuyée sur du code

Pourquoi Python plutôt qu'un autre outil

- Gratuit, open source (vs Matlab, SPSS, ArcGIS)
- Un seul langage pour scraper, nettoyer, analyser, cartographier, publier
- Écosystème et communauté très larges, énormément de tutoriels
- Notebooks = bon compromis code/narration, adapté à un public non-développeur

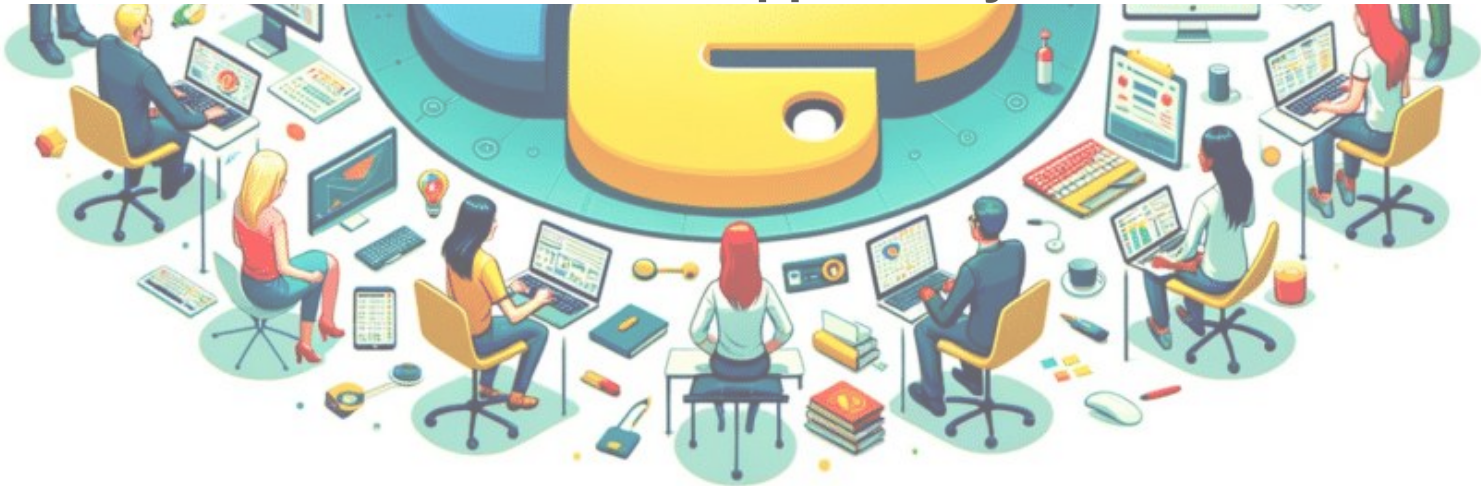
Et R ?

Une communauté R tout aussi active existe en SHS quantitatives (sociologie, démographie, psychologie, économétrie), historiquement forte sur les stats avancées et la visualisation (tidyverse, ggplot2). Le choix dépend souvent de la discipline et des habitudes des pairs — Python et R sont complémentaires plus que concurrents, et interopérables (reticulate, rpy2).



**Python seul est relativement simple.
Python + son écosystème, c'est énorme.**

≈ 25 millions d'utilisateurs/développeurs Python dans le monde



Syntax error... Et Python dans tout ça ?

```
</> asm

mov eax, 28      ; température = 28
cmp eax, 25      ; comparer température à 25
jle fin         ; si température <= 25, aller à "fin"

; sinon afficher "Il fait chaud"
lea rdi, message
call puts

fin:
    ret

message:
    db "Il fait chaud", 0
```

```
</> Java

public class Main {
    public static void main(String[] args) {
        int temperature = 28;

        if (temperature > 25) {
            System.out.println("Il fait chaud");
        }
    }
}
```

```
</> C

#include <stdio.h>

int main() {
    int temperature = 28;

    if (temperature > 25) {
        printf("Il fait chaud\n");
    }

    return 0;
}
```

```
</> Python

temperature = 28

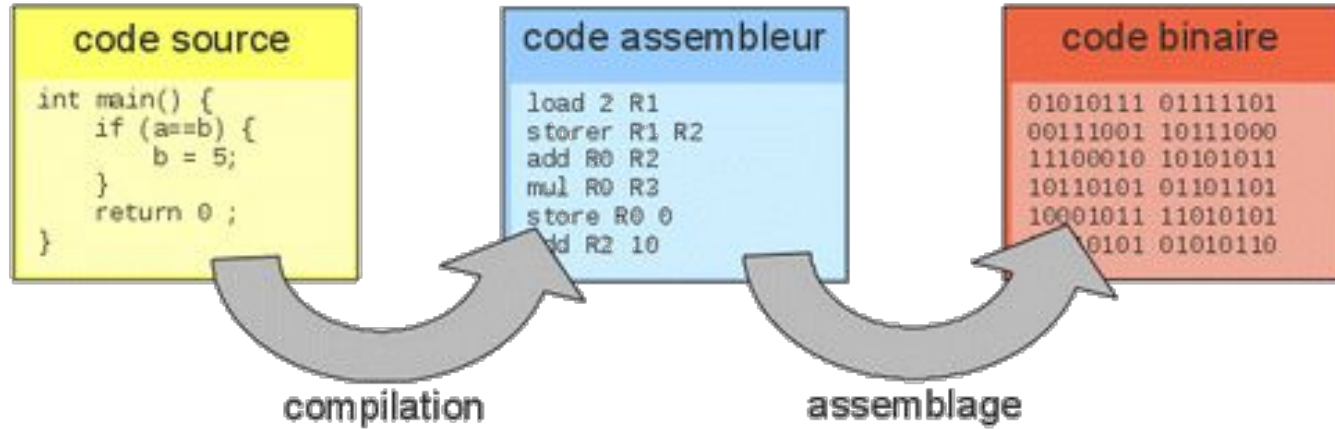
if temperature > 25:
    print("Il fait chaud")
```

Python a été conçu avec une idée assez agréable :

le code doit rester lisible par les humains.

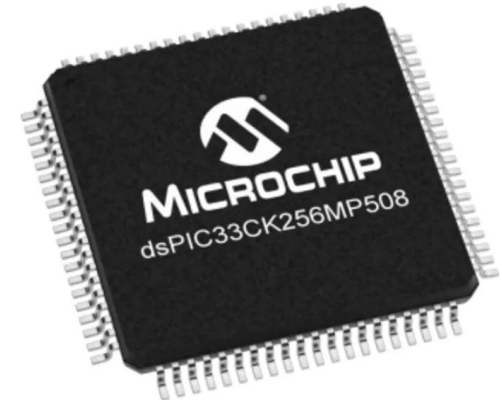
Même sans connaître Python, on peut probablement deviner ce que fait ce programme.

Langages **compilés** / interprétés

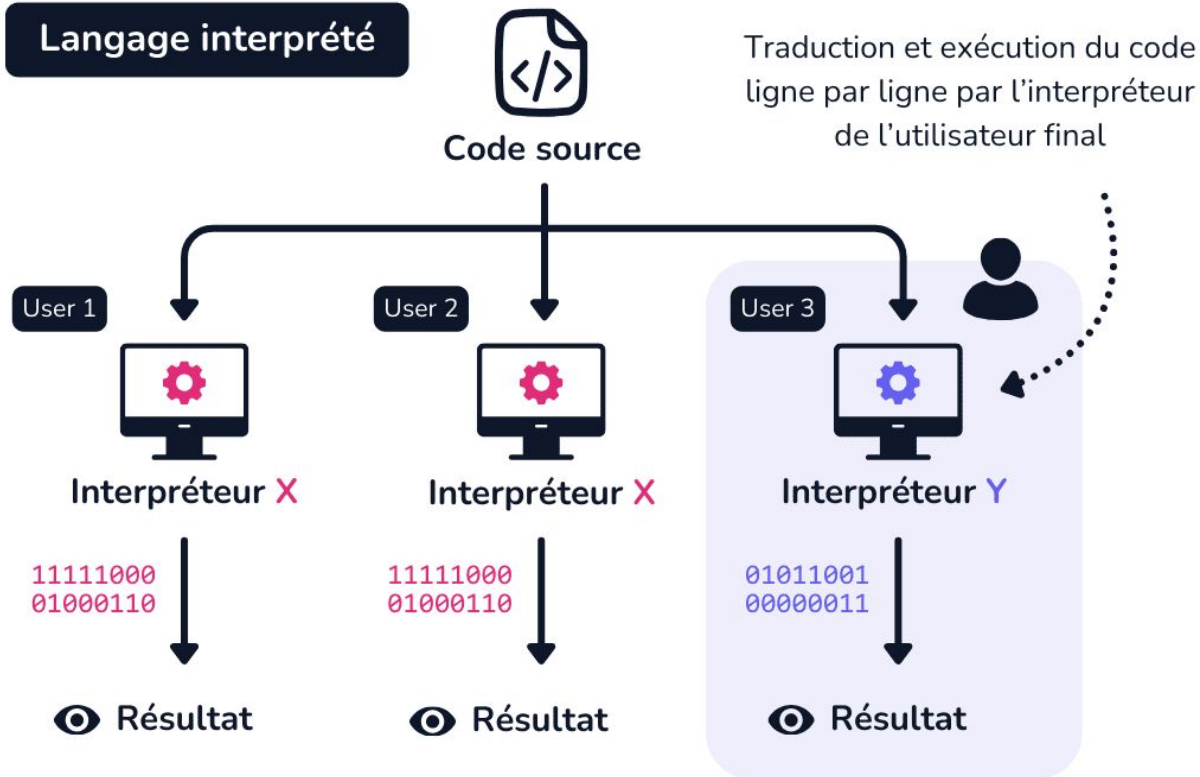


```
1 #include <stdio.h>  
2  
3 #define SUCCESS 0  
4  
5 int somme(int e1, int e2)  
6 {  
7     return (e1 + e2);  
8 }  
9  
10 int main(int argc, char const *argv[])  
11 {  
12     int a=4, b=5;  
13     printf("%d\n", somme(a, b));  
14     return SUCCESS;  
15 }
```

Avantages ?
Inconvénients ?

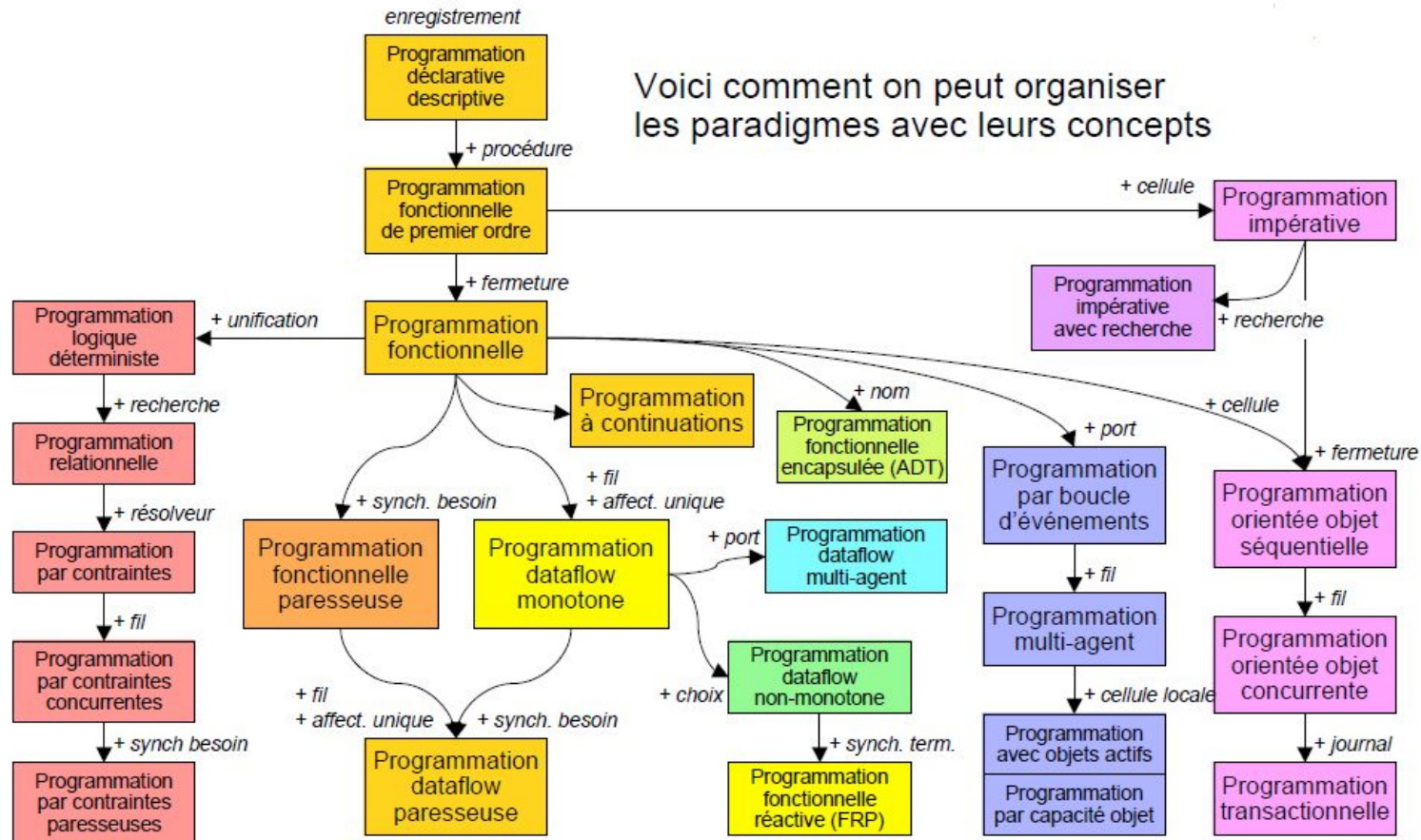


Langages compilés / interprétés



Avantages ?
Inconvénients ?

Paradigmes et langages de programmation



Paradigmes et langages de programmation

Paradigm	Languages
Functional	Erlang, F#, Haskell, Lisp, Ocaml, Perl, Racket, Ruby, Rust;
Imperative	Ada, C, C++, F#, Fortran, Go, Ocaml, Pascal, Rust;
Object-Oriented	Ada, C++, C#, Chapel, Dart , F#, Java, JavaScript, Ocaml, Perl, PHP, Python, Racket, Rust, Smalltalk, Swift, TypeScript;
Scripting	Dart, Hack, JavaScript, JRuby, Lua, Perl, PHP, Python, Ruby, TypeScript;

Un langage de script est un langage de programmation interprété (sans compilation explicite), conçu pour manipuler les fonctionnalités d'un système informatique. Sa syntaxe de haut niveau permet de s'affranchir des contraintes des commandes de bas niveau. Ses capacités dépendent de l'environnement et de l'interface de son interpréteur.

La **programmation fonctionnelle** est un paradigme de programmation de type déclaratif.

Elle conçoit le calcul comme l'évaluation de fonctions mathématiques.

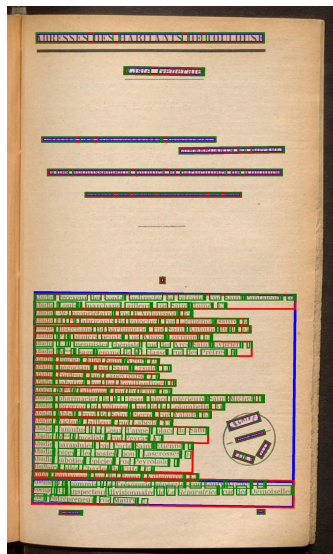
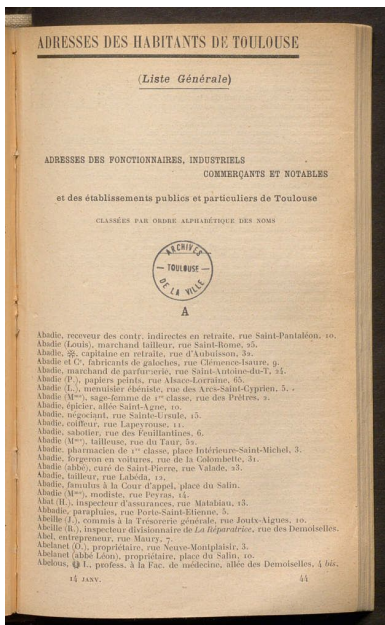
La **programmation impérative** est un paradigme informatique qui définit les opérations par des séquences d'instructions. Ces instructions sont exécutées par l'ordinateur afin de modifier l'état du programme.

C'est la forme de programmation la plus courante. Elle s'oppose à la **programmation déclarative**, qui inclut notamment la programmation logique et la programmation fonctionnelle.

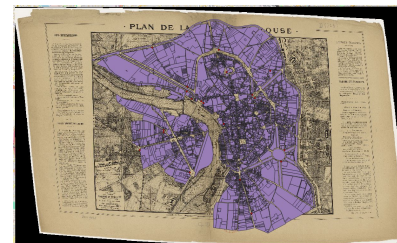
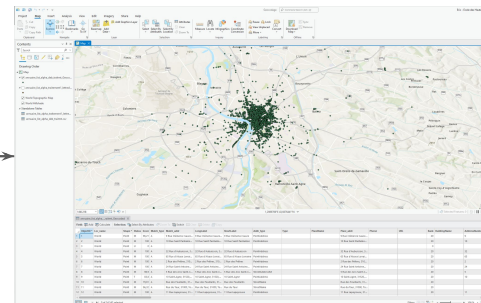
Le paradigme de la **programmation orientée objet**, ou programmation par objet, est une approche fondamentale de la programmation informatique :

- Un objet modélise une entité du monde réel (ex : voiture, personne) ou conceptuelle.
- Représenter ces objets et leurs relations.
- Leur interaction permet de concevoir les fonctionnalités et d'optimiser la résolution des problèmes.

Exemple 1 : Annuaire Toulouse 1900 - OCR, nettoyage, géocodage



```
{
  "textAnnotations": [
    {
      "locale": "fr",
      "description": "ADRESSES DES HABITANTS DE TOULOUSE",
      "boundingPoly": {
        "vertices": [
          {
            "x": 124,
            "y": 136
          },
          {
            "x": 1502,
            "y": 136
          },
          {
            "x": 1502,
            "y": 2644
          },
          {
            "x": 124,
            "y": 2644
          }
        ]
      },
      "description": "ADRESSES",
      "boundingPoly": {
        "vertices": [
          {
            "x": 148,
            "y": 136
          },
          {
            "x": 471,
```



Exemples 2 :
Applications web PTM



Interrogation d'une base de données avec des expressions régulières

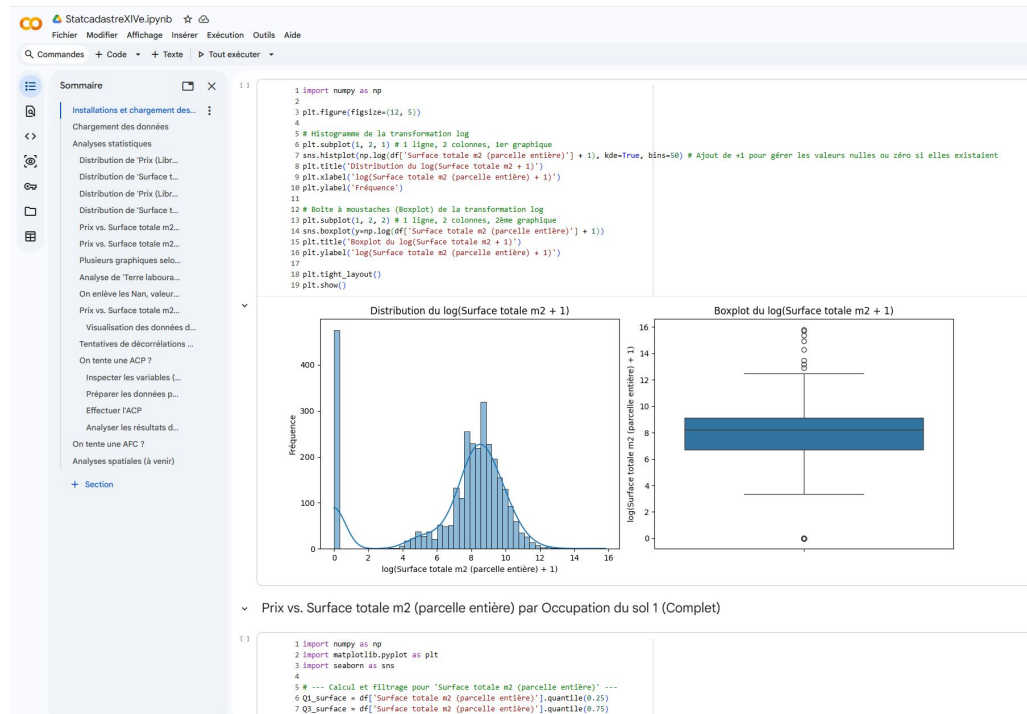


Géoréférencement de cartes anciennes et tuilage



Calcul de plus courts chemins

Exemples 3 : Carnet / Notebook Jupyter



Analyses statistiques & spatiales / collaboratives

https://colab.research.google.com/drive/1F3BmNGHl6nZj2D-7_aGPeOfEfdonmV

Publi : <https://jhnr.net/articles/10.25517/jhnr.v10i1.86>

Exemples 4 : Webscraping

yahoo! finance Rechercher des actualités, des symboles ou des entreprises

Accueil Finance Mon Portfeuille Panorama du marché Bourse Finance Perso Secteurs Convertisseur High Tech

La bourse ferme dans 6 h 45 min

CAC 40
7687,96
+55,88 (+0,73 %)

Euro Stoxx 50
4 987,05
+21,25 (+0,43 %)

Dow Jones
39 308,00
-23,00 (-0,06 %)

EUR/USD
1,0799
+0,0014 (+0,1326 %)

Gold future
2 369,40
0,00 (0,00 %)

Bitcoin EUR
53 363,18
-2 877,84 (-5,12 %)

« La victoire est encore possible si vous nous en donnez les moyens » : pourquoi l'Ukraine... »
Malgré un front stabilisé, l'Ukraine souffre de la guerre énergétique et craint une nouvelle donne à l'Ouest. Entre montée populiste en Europe et retour de Trump à...

Mes portefeuilles et marchés
Récemment consulté >

Symbole	Dernier cours	Variation	% variation
*FCHI	7687,96	+55,88	+0,73 %
CAC 40			

Mes titres à surveiller
Connectez-vous pour consulter votre liste et ajouter des symboles.

Crypto-monnaies >

Symbole	Dernier cours	Variation	% variation
BTC-EUR	53 363,18	-2 877,84	-5,12 %
ETH-EUR	2 933,01	-163,15	-5,27 %
USD...	0,9251	+0,0003	+0,0374 %
BNB-E...	491,74	-32,36	-6,17 %
SOL-EUR	124,39	-10,93	-8,08 %

Devises >

Symbole	Dernier cours	Variation	% variation
EURUSD...	1,0799	+0,0014	+0,1326 %
EURGBP...	0,8468	+0,0008	+0,0910 %
EURCHF...	0,9736	+0,0018	+0,1811 %
EURCAD...	1,4711	+0,0005	+0,0320 %
EURJPY...	7,8500	+0,0096	+0,12 %

En vogue >

Symbole	Dernier cours	Variation	% variation
*FCHI	7687,96	+55,88	+0,73 %
CAC 40			

Actions : Plus fortes >

Symbole	Dernier cours	Variation	% variation
AI R&F PA	0,0004	0,0000	0,00 %

Entreprise Business Wire • il y a 1 heure
PHONE WEB – RESULTATS ANNUELS 2023
PARIS, July 04, 2024—Regulatory News: 1. CHIFFRES CLES

Techno IPR Newswire • il y a 1 heure
BLUE FROG ROBOTICS : succès d'une pépite de la French Tech qui met la technologie au service des enfants malades
Chaque année, près de 11 000 élèves atteints d'une maladie chronique ou de longue durée sont scolarisés dans les établissements hospitaliers et scolaires, selon le Ministère de l'Éducation Nationale (1)

Python script

http request



```
1 url = 'https://www.tse-fr.eu/' # Remplacez par l'URL que vous souhaitez scraper
2 response = requests.get(url)
3
4
5 # Vérifions que la requête a réussi
6 if response.status_code == 200:
7     print("Page récupérée avec succès")
8 else:
9     print("Échec de la récupération de la page")
```

BeautifulSoup



```
1 soup = BeautifulSoup(response.content, 'html.parser')
```



Cemantix Breaker



spaCy

Industrial-Strength Natural Language Processing

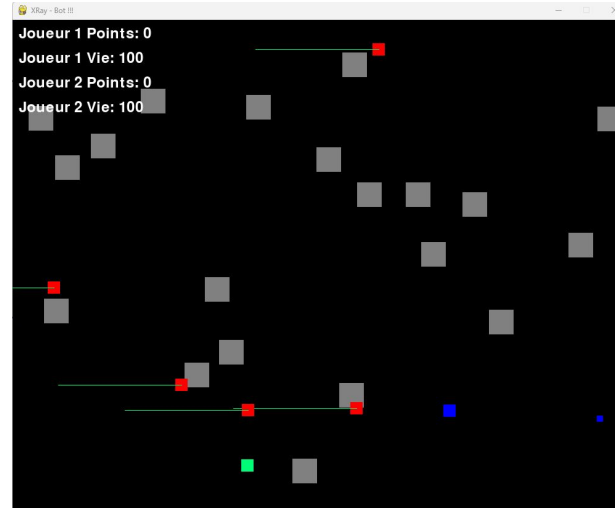
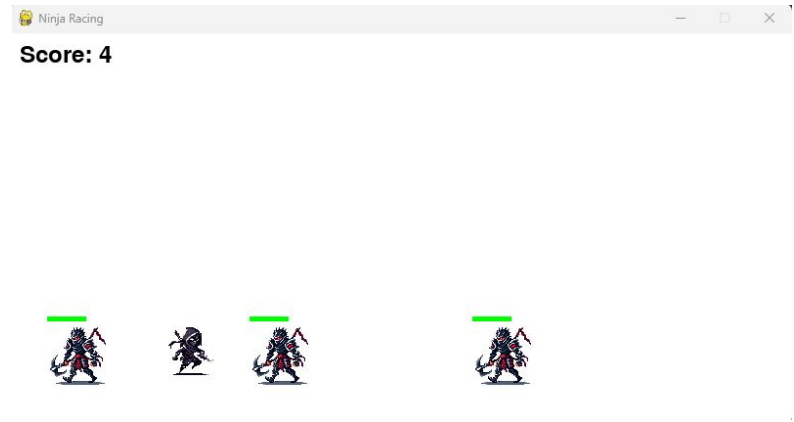
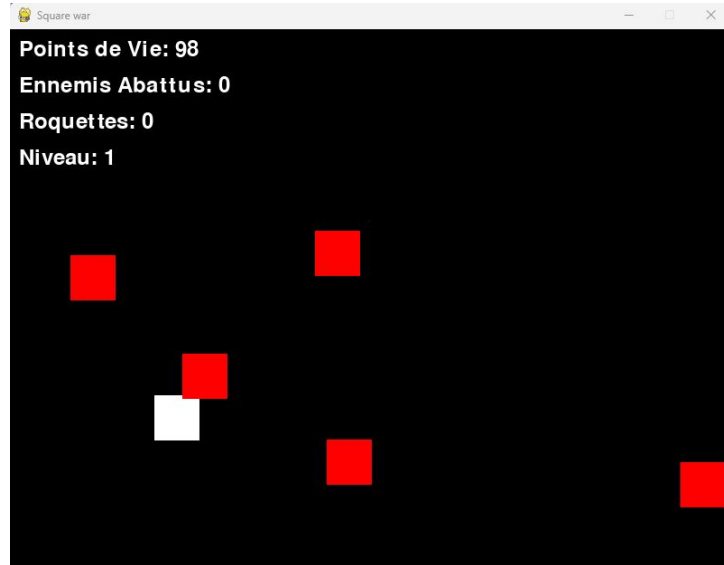
<https://spacy.io/>



<https://cemantix.certitudes.org/>

Exemple 5

Jeux vidéo



Route 1 : installer et configurer VS Code

The screenshot displays the Visual Studio Code (VS Code) interface with a dark theme. The Explorer sidebar on the left shows a project structure with folders like 'PYTHON SIS' and '1_manipulations_basiques.py'. The main editor area is open to a file named '1_manipulations_basiques.py', which contains Python code for creating and writing to a CSV file. The code includes comments in French and uses the 'csv' module. The output console at the bottom shows the execution results, including the message 'Fichier CSV créé !'. On the right side, there is a 'CONVERSATION' panel with a 'RECENT SESSIONS' list and a 'Build with Agent' section. The 'Build with Agent' section includes a note about response accuracy and a link to 'Generate Agent Instructions'. At the bottom right, there is a 'Cellule 50 sur 65' indicator and a 'Continue (N5)' button.

```
import csv

# Ecrivez dans un fichier CSV
with open('csv/nouveau_fichier.csv', 'w', newline='') as fichier_csv:
    ecrivain_csv = csv.writer(fichier_csv)
    ecrivain_csv.writerow(['nom', 'age', 'ville'])
    ecrivain_csv.writerow(['Alice', 25, 'Paris'])
    ecrivain_csv.writerow(['Bob', 30, 'Lyon'])

print("Fichier CSV créé !")
```

Fichier CSV créé !

Exercices sur les fichiers

Exercice 1 : Boucle et fichier texte

Créez un programme qui lit le fichier texte `mails_exemple.txt` ligne par ligne et affiche chaque ligne à l'écran. Si le fichier contient le mot "important", affichez une alerte particulière.

Exercice 2 : Manipulation de CSV

Créez un programme qui lit un fichier CSV contenant une liste de produits (nom, prix, quantité) et qui calcule la valeur totale des stocks (prix * quantité).

```
# Exercice 1 : Lecture avec recherche de mot
```

```
# Exercice 2 : Calcul de valeur totale
```

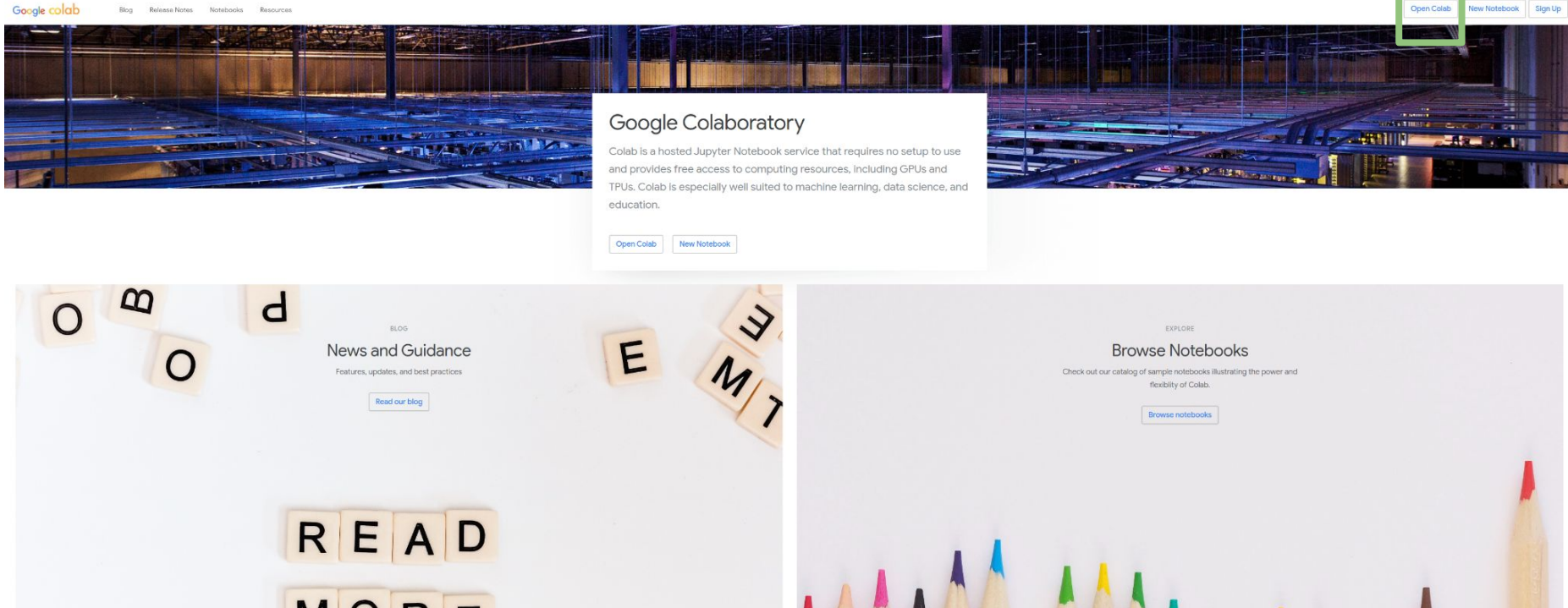
Résumé

Dans cette première séance, nous avons couvert :

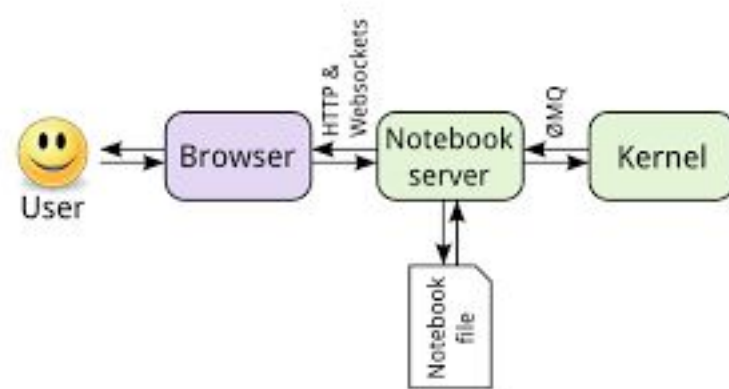
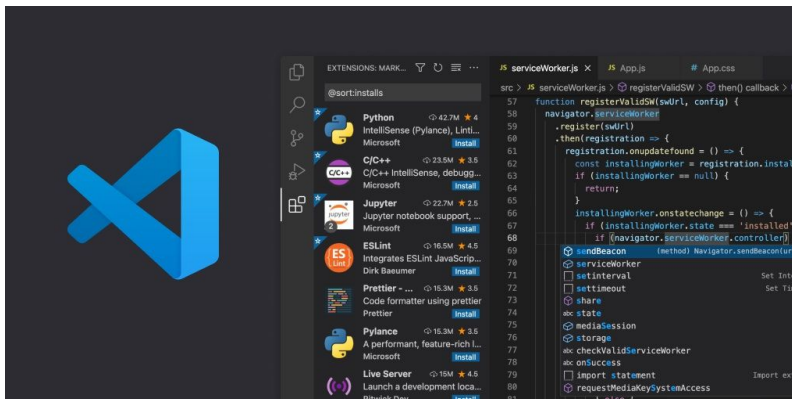
1. **Introduction à la programmation** : langages compilés vs interprétés
2. **Variables et types** : int, float, str, bool
3. **Opérations** : arithmétiques, booléennes, sur les chaînes
4. **Formatage** : f-strings et slicing
5. **Conditions** : if/elif/else
6. **Boucles** : for, while, enumerate, zip
7. **Fonctions** : définition, paramètres, retour
8. **Gestion d'erreurs** : try/except

Route 2 : Google Colab

Aller à l'adresse : <https://colab.google/>



Exécution de programmes Python : local vs. distant



IDE : integrated development environment

VS code

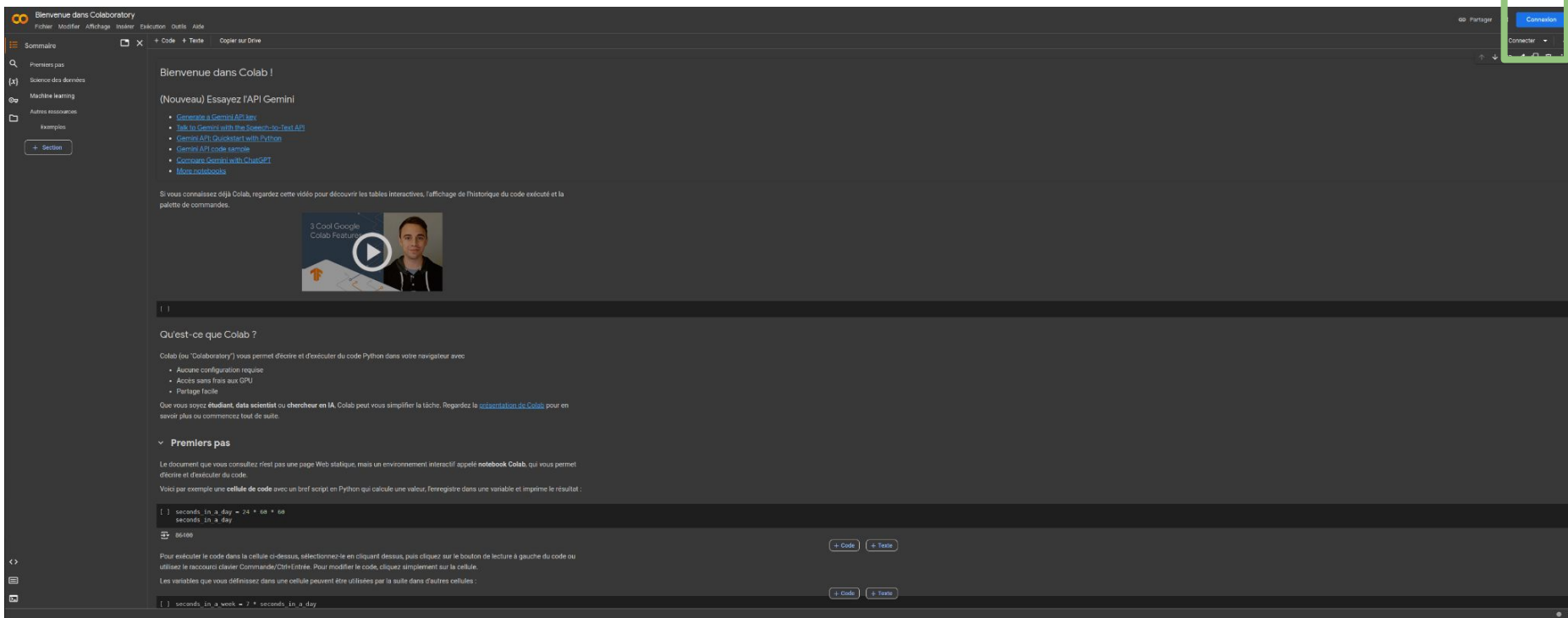
PyCharm

Eclipse

...

Dans un navigateur ?

Connexion avec un compte gmail



Bienvenue dans Colaboratory

Fichier Modifier Affichage Insérer Exécution Outils Aide

Sommaire

Prémiers pas

Science des données

Machine learning

Autres ressources

Exemples

+ Section

Bienvenue dans Colab !

(Nouveau) Essayez l'API Gemini

- [Introduction à Gemini API](#)
- [Hello to Gemini with the Jupyter-Notebook API](#)
- [Gemini API Quickstart with Python](#)
- [Gemini API code sample](#)
- [Compare Gemini with ChatGPT](#)
- [More notebooks](#)

Si vous connaissez déjà Colab, regardez cette vidéo pour découvrir les tables interactives, l'affichage de l'historique du code exécuté et la palette de commandes.

3 Cool Google Colab Features

[]

Qu'est-ce que Colab ?

Colab (ou "Colaboratory") vous permet d'écrire et d'exécuter du code Python dans votre navigateur avec :

- Aucune configuration requise
- Accès sans frais aux GPU
- Partage facile

Que vous soyez **étudiant**, **data scientist** ou **chercheur en IA**, Colab peut vous simplifier la tâche. Regardez la [représentation de Colab](#) pour en savoir plus ou commencez tout de suite.

▼ Premiers pas

Le document que vous consultez, n'est pas une page Web statique, mais un environnement interactif appelé **notebook Colab**, qui vous permet d'écrire et d'exécuter du code.

Voici par exemple une **cellule de code** avec un bref script en Python qui calcule une valeur, l'enregistre dans une variable et imprime le résultat :

```
[ ] seconds, in a day = 24 * 60 * 60
seconds, in a day
```

86400

Pour exécuter le code dans la cellule ci-dessus, sélectionnez-le en cliquant dessus, puis cliquez sur le bouton de lecture à gauche du code ou utilisez le raccourci clavier Commande/Ctrl+Entrée. Pour modifier le code, cliquez simplement sur la cellule.

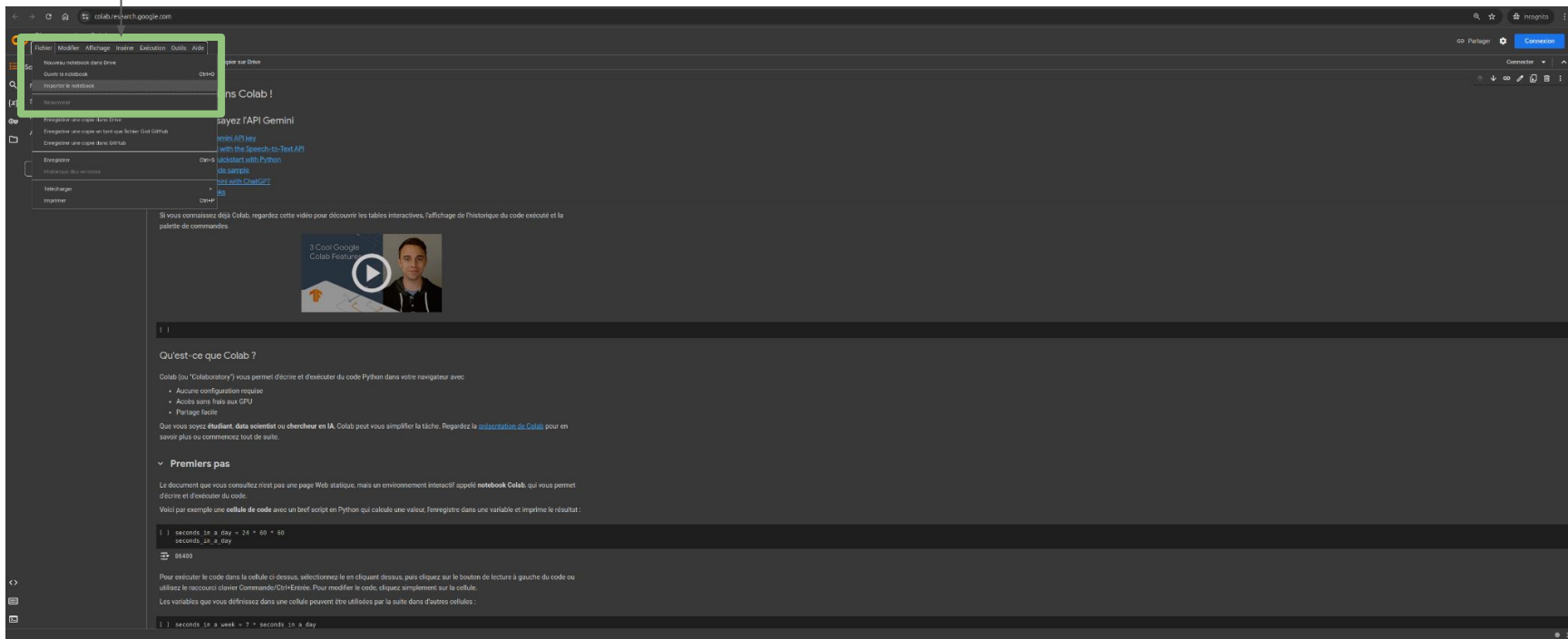
Les variables que vous définissez dans une cellule peuvent être utilisées par la suite dans d'autres cellules :

```
[ ] seconds, in a week = 7 * seconds, in a day
```

+ Code -> Texte

+ Code -> Texte

Fichier / Importer le notebook



The screenshot shows the Google Colab web interface. The 'Fichier' (File) menu is open, and the 'Importer le notebook' (Import notebook) option is highlighted. The interface is in French. The main content area displays a welcome message and a video player. Below the video, there is a section titled 'Qu'est-ce que Colab ?' (What is Colab?) which explains that Colab is an interactive environment for running Python code. It lists features such as no configuration required, free access to GPUs, and easy sharing. A 'Premiers pas' (First steps) section follows, explaining that the document is an interactive notebook and providing instructions on how to execute code cells. The code cell shown contains a simple calculation: `seconds_in_a_day = 24 * 60 * 60` and `seconds_in_a_week = 7 * seconds_in_a_day`.

colab.research.google.com

Fichier Modifier Affichage Insérer Extension Outils Aide

Nouveaux notebooks dans Drive

Ouvrir le notebook

Importer le notebook

Rechercher

Envoyer une copie dans Drive

Envoyer une copie en tant que fichier Google Docs

Envoyer une copie dans VM Lab

Envoyer

Historique des versions

Télécharger

Imprimer

ins Colab !

Essayez l'API Gemini

with API key

with the Speed-to-Text API

kickstart with Python

de sample

not with ChatGPT

Si vous connaissez déjà Colab, regardez cette vidéo pour découvrir les tables interactives, l'affichage de l'historique du code exécuté et la palette de commandes.

3 Cool Google Colab Features

Qu'est-ce que Colab ?

Colab (ou "Colaboratory") vous permet d'écrire et d'exécuter du code Python dans votre navigateur avec :

- Aucune configuration requise
- Accès sans frais aux GPU
- Partage facile

Que vous soyez étudiant, data scientist ou chercheur en IA, Colab peut vous simplifier la tâche. Regardez le [tutorial de Colab](#) pour en savoir plus ou commencez tout de suite.

▼ Premiers pas

Le document que vous consultez n'est pas une page Web statique, mais un environnement interactif appelé **notebook Colab**, qui vous permet d'écrire et d'exécuter du code.

Voici par exemple une **cellule de code** avec un bref script en Python qui calcule une valeur, l'enregistre dans une variable et imprime le résultat :

```
1 | seconds_in_a_day = 24 * 60 * 60
   seconds_in_a_day
   86400
```

Pour exécuter le code dans la cellule ci-dessus, sélectionnez-le en cliquant dessus, puis cliquez sur le bouton de lecture à gauche du code ou utilisez le raccourci clavier Commande/Ctrl+Entrée. Pour modifier le code, cliquez simplement sur la cellule.

Les variables que vous définissez dans une cellule peuvent être utilisées par la suite dans d'autres cellules :

```
1 | seconds_in_a_week = 7 * seconds_in_a_day
```


Dans un autre onglet, aller à l'adresse :
<https://github.com/ericmermet/python-shs>

The screenshot shows the GitHub repository page for 'python-tut' by user 'ericmermet'. The repository is public and has 1 branch and 0 tags. The main branch is 'main'. The repository contains several files and folders, including 'csv', 'Python1_TUT_INNO3.ipynb', 'maia_examples.txt', and 'nouveau_fichier.txt'. The repository also has a README file, which is currently empty and prompts the user to 'Add a README'. The repository has 0 stars, 1 watching, and 0 forks. The repository is created on 01/17/2024, 25 minutes ago. The repository is currently empty, with a message 'No description, website, or topics provided.' and a button to 'Create a new release'. The repository also has a section for 'Packages' and 'Languages', both of which are currently empty.

ericmermet / python-tut

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

python-tut PUBLIC

main 1 Branch 0 Tags

Go to file Add file Code

ericmermet Delete correction_exercice_final.ipynb 01/17/2024 25 minutes ago 0 Commits

- csv data structures / example files 7 hours ago
- Python1_TUT_INNO3.ipynb add final exercice 27 minutes ago
- maia_examples.txt data structures / example files 7 hours ago
- nouveau_fichier.txt data structures / example files 7 hours ago

README

Add a README

Help people interested in this repository understand your project by adding a README.

Add a README

About

No description, website, or topics provided.

Activity

- 0 stars
- 1 watching
- 0 forks

Releases

No releases published

Create a new release

Packages

No packages published

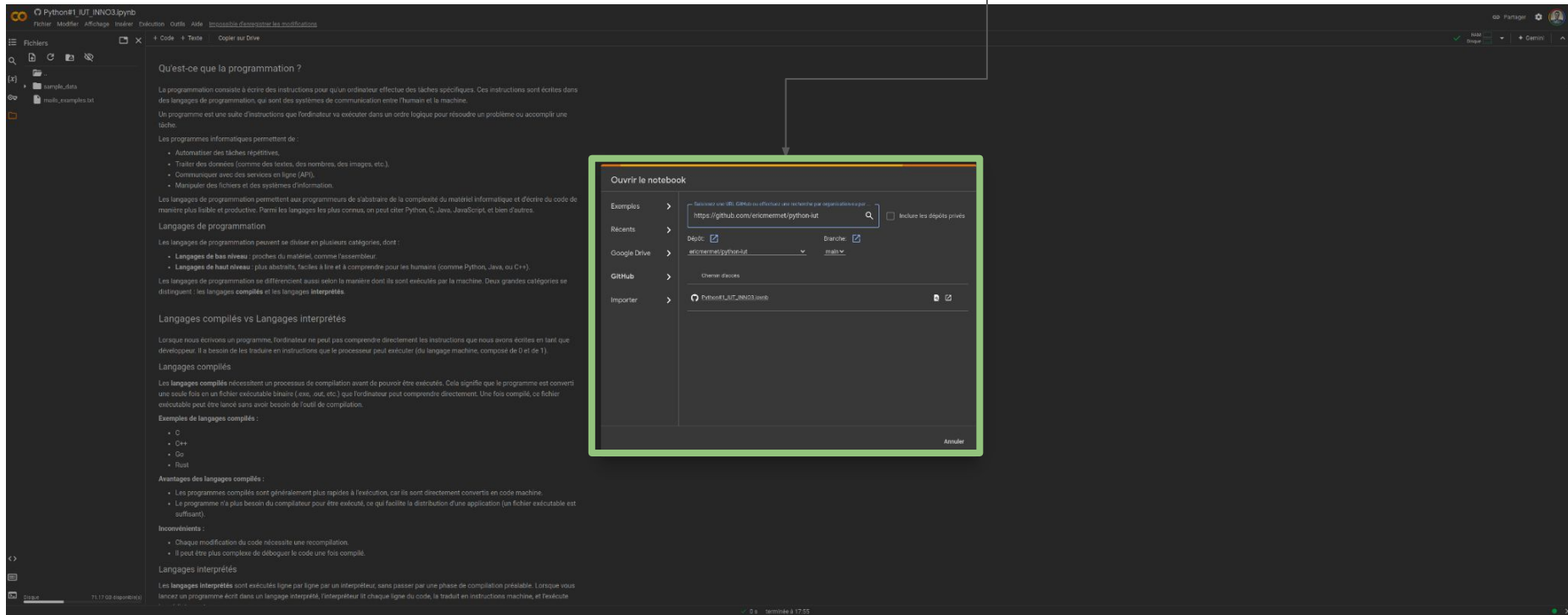
Publish your first package

Languages

- Jupyter Notebook 100.0%

© 2024 GitHub, Inc. Terms Privacy Security Status Docs Contact Manage cookies Do not share my personal information

Revenir sur le google collabs et entrer l'adresse du git dans la barre



The screenshot shows a JupyterLab environment. On the left is a file browser with folders like 'sample_data' and 'notebooks'. The main area displays a document titled 'Qu'est-ce que la programmation ?' with text about programming and a list of languages. A modal window titled 'Ouvrir le notebook' is open in the foreground, showing a search bar with the URL 'https://github.com/enicemmet/python-ut', a 'Depôt' dropdown set to 'github.com', and a list of repositories including 'python-ut'. A green box highlights the modal window, and a line from the text above points to the search bar.

Qu'est-ce que la programmation ?

La programmation consiste à écrire des instructions pour qu'un ordinateur effectue des tâches spécifiques. Ces instructions sont écrites dans des langages de programmation, qui sont des systèmes de communication entre l'humain et la machine.

Un programme est une suite d'instructions que l'ordinateur va exécuter dans un ordre logique pour résoudre un problème ou accomplir une tâche.

Les programmes informatiques permettent de :

- Automatiser des tâches répétitives.
- Traiter des données (comme des textes, des nombres, des images, etc.).
- Communiquer avec des services en ligne (API).
- Gérer des fichiers et des systèmes d'information.

Les langages de programmation permettent aux programmeurs de s'abstraire de la complexité du matériel informatique et d'écrire du code de manière plus lisible et productive. Parmi les langages les plus connus, on peut citer Python, C, Java, JavaScript, et bien d'autres.

Langages de programmation

Les langages de programmation peuvent se diviser en plusieurs catégories, dont :

- Langages de bas niveau : proches du matériel, comme l'assembleur.
- Langages de haut niveau : plus abstraits, faciles à lire et à comprendre pour les humains (comme Python, Java, ou C++).

Les langages de programmation ne diffèrent aussi selon la manière dont ils sont exécutés par la machine. Deux grandes catégories se distinguent : les langages **compilés** et les langages **interprétés**.

Langages compilés vs Langages interprétés

Lorsque nous écrivons un programme, l'ordinateur ne peut pas comprendre directement les instructions que nous avons écrites en tant que développeur. Il a besoin de les traduire en instructions que le processeur peut exécuter (du langage machine, composé de 0 et de 1).

Langages compilés

Les **langages compilés** nécessitent un processus de compilation avant de pouvoir être exécutés. Cela signifie que le programme est converti une seule fois en un fichier exécutable binaire (.exe, .out, etc.) que l'ordinateur peut comprendre directement. Une fois compilé, ce fichier exécutable peut être lancé sans avoir besoin de l'outil de compilation.

Exemples de langages compilés :

- C
- C++
- Go
- Rust

Avantages des langages compilés :

- Les programmes compilés sont généralement plus rapides à l'exécution, car ils sont directement convertis en code machine.
- Le programme n'a plus besoin du compilateur pour être exécuté, ce qui facilite la distribution d'une application (un fichier exécutable est suffisant).

Inconvénients :

- Chaque modification du code nécessite une recompilation.
- Il peut être plus complexe de déboguer le code une fois compilé.

Langages interprétés

Les **langages interprétés** sont exécutés ligne par ligne par un interpréteur, sans passer par une phase de compilation préalable. Lorsque vous lancez un programme écrit dans un langage interprété, l'interpréteur lit chaque ligne du code, le traduit en instructions machine, et l'exécute.

Ouvrir le notebook

Exemples > Recherchez un dépôt GitHub ou effectuez une recherche par organisation ou par https://github.com/enicemmet/python-ut

Récents >

Google Drive > Dépot: github.com Branche: main

GitHub > Chercher des repos

Importer > python-ut

Annuler

71.17.028 (rapporté)

0 s terminée à 17:55

Revenir sur le google collabs et entrer l'adresse du git dans la barre

Python LUT - JupyterLab

Fichier Modifier Affichage Insérer Outils Aide [Accéder à l'aperçu des modifications](#)

Fichiers

Qu'est-ce que la programmation ?

La programmation consiste à écrire des instructions pour qu'un ordinateur effectue des tâches spécifiques. Ces instructions sont écrites dans des langages de programmation, qui sont des systèmes de communication entre l'humain et la machine.

Un programme est une suite d'instructions que l'ordinateur va exécuter dans un ordre logique pour résoudre un problème ou accomplir une tâche.

Les programmes informatiques permettent de :

- Automatiser des tâches répétitives.
- Traiter des données (comme des textes, des nombres, des images, etc.).
- Communiquer avec des services en ligne (API).
- Gérer des fichiers et des systèmes d'information.

Les langages de programmation permettent aux programmeurs de s'abstraire de la complexité du matériel informatique et d'écrire du code de manière plus lisible et productive. Parmi les langages les plus connus, on peut citer Python, C, Java, JavaScript, et bien d'autres.

Langages de programmation

Les langages de programmation peuvent se diviser en plusieurs catégories, dont :

- Langages de bas niveau : proches du matériel, comme l'assembleur.
- Langages de haut niveau : plus abstraits, faciles à lire et à comprendre pour les humains (comme Python, Java, ou C++).

Les langages de programmation ne diffèrent aussi selon la manière dont ils sont exécutés par la machine. Deux grandes catégories se distinguent : les langages **compilés** et les langages **interprétés**.

Langages compilés vs Langages interprétés

Lorsque nous écrivons un programme, l'ordinateur ne peut pas comprendre directement les instructions que nous avons écrites en tant que développeur. Il a besoin de les traduire en instructions que le processeur peut exécuter (du langage machine, composé de 0 et de 1).

Langages compilés

Les **langages compilés** nécessitent un processus de compilation avant d'être exécutés. Cela signifie que le programme est converti une seule fois en un fichier exécutable binaire (.exe, .out, etc.) que l'ordinateur peut comprendre directement. Une fois compilé, ce fichier exécutable peut être lancé sans avoir besoin de l'outil de compilation.

Exemples de langages compilés :

- C
- C++
- Go
- Rust

Avantages des langages compilés :

- Les programmes compilés sont généralement plus rapides à l'exécution, car ils sont directement convertis en code machine.
- Le programme n'a plus besoin du compilateur pour être exécuté, ce qui facilite la distribution d'une application (un fichier exécutable est suffisant).

Inconvénients :

- Chaque modification du code nécessite une recompilation.
- Il peut être plus complexe de déboguer le code une fois compilé.

Langages interprétés

Les **langages interprétés** sont exécutés ligne par ligne par un interpréteur, sans passer par une phase de compilation préalable. Lorsque vous lancez un programme écrit dans un langage interprété, l'interpréteur lit chaque ligne du code, le traduit en instructions machine, et l'exécute.

Ouvrir le notebook

Exemples > Recherchez une URL GitHub ou effectuez une recherche par organisation ou par <https://github.com/enicomet/python-lut> ☐ Inclure les dépôts privés

Récents >

Dépot: ☐ Branche:

Google Drive >

GitHub >

Importer >

0 s terminée à 17:55

Revenir sur le google collabs et entrer l'adresse du git dans la barre

Python IDE - JupyterLab

Fichier Modifier Affichage Insérer Extension Outils Aide [Accéder à l'interface des modifications](#)

Fichiers

Code Texte Copier sur Drive

Qu'est-ce que la programmation ?

La programmation consiste à écrire des instructions pour qu'un ordinateur effectue des tâches spécifiques. Ces instructions sont écrites dans des langages de programmation, qui sont des systèmes de communication entre l'humain et la machine.

Un programme est une suite d'instructions que l'ordinateur va exécuter dans un ordre logique pour résoudre un problème ou accomplir une tâche.

Les programmes informatiques permettent de :

- Automatiser des tâches répétitives.
- Traiter des données (comme des textes, des nombres, des images, etc.).
- Communiquer avec des services en ligne (API).
- Gérer des fichiers et des systèmes d'information.

Les langages de programmation permettent aux programmeurs de s'abstraire de la complexité du matériel informatique et d'écrire du code de manière plus lisible et productive. Parmi les langages les plus connus, on peut citer Python, C, Java, JavaScript, et bien d'autres.

Langages de programmation

Les langages de programmation peuvent se diviser en plusieurs catégories, dont :

- Langages de bas niveau : proches du matériel, comme l'assembleur.
- Langages de haut niveau : plus abstraits, faciles à lire et à comprendre pour les humains (comme Python, Java, ou C++).

Les langages de programmation ne diffèrent aussi selon la manière dont ils sont exécutés par la machine. Deux grandes catégories se distinguent : les langages **compilés** et les langages **interprétés**.

Langages compilés vs Langages interprétés

Lorsque nous écrivons un programme, l'ordinateur ne peut pas comprendre directement les instructions que nous avons écrites en tant que développeur. Il a besoin de les traduire en instructions que le processeur peut exécuter (du langage machine, composé de 0 et de 1).

Langages compilés

Les **langages compilés** nécessitent un processus de compilation avant d'être exécutés. Cela signifie que le programme est converti une seule fois en un fichier exécutable binaire (.exe, .out, etc.) que l'ordinateur peut comprendre directement. Une fois compilé, ce fichier exécutable peut être lancé sans avoir besoin de l'outil de compilation.

Exemples de langages compilés :

- C
- C++
- Go
- Rust

Avantages des langages compilés :

- Les programmes compilés sont généralement plus rapides à l'exécution, car ils sont directement convertis en code machine.
- Le programme n'a plus besoin du compilateur pour être exécuté, ce qui facilite la distribution d'une application (un fichier exécutable est suffisant).

Inconvénients :

- Chaque modification du code nécessite une recompilation.
- Il peut être plus complexe de déboguer le code une fois compilé.

Langages interprétés

Les **langages interprétés** sont exécutés ligne par ligne par un interpréteur, sans passer par une phase de compilation préalable. Lorsque vous lancez un programme écrit dans un langage interprété, l'interpréteur lit chaque ligne du code, le traduit en instructions machine, et l'exécute.

Ouvrir le notebook

Exemples > Recherchez une URL GitHub ou effectuez une recherche par organisation ou par [https://github.com/enicomet/python-ut](#) ☐ Inclure les dépôts privés

Récents >

Dépot: Branche:

Google Drive >

GitHub >

Importer >

0 s terminée à 17:55